UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Dipartimento
di Fisica
e Astronomia
Galileo Galilei

No U-Turn Sampler

Adaptively Setting Path Lengths in Hamiltonian Monte Carlo

Pieripolli Leonardo, Pirazzo Tommaso,
Ponchio Mattia, Secco Benedetto

August 21, 2026

No U-Turn Sampler

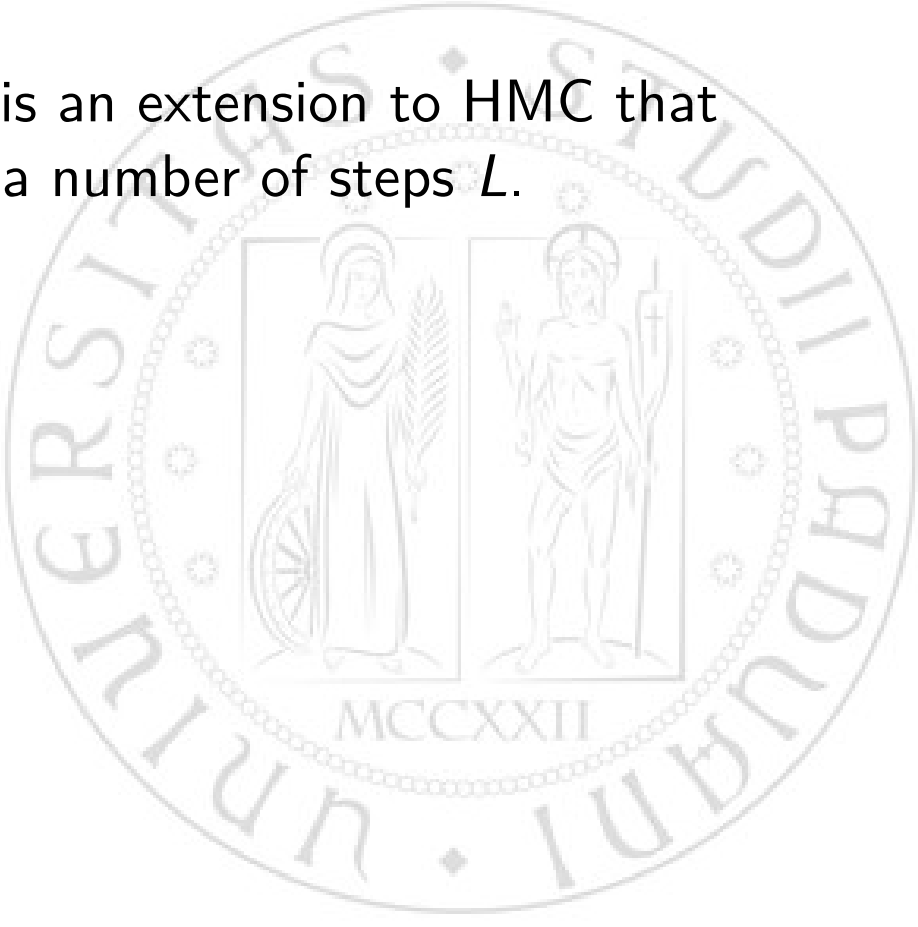
HMC is an *MCMC* algorithm that avoids random walk behavior and sensitivity to correlated parameters.
However, it requires the tuning of two user-specified parameters: step size ϵ and number of steps L.

No U-Turn Sampler

HMC is an *MCMC* algorithm that avoids random walk behavior and sensitivity to correlated parameters.
However, it requires the tuning of two user-specified parameters: step size ϵ and number of steps L.

$$\Downarrow$$

The *No-U-Turn Sampler* (NUTS) is an extension to HMC that eliminates the need to set a number of steps L.



No U-Turn Sampler

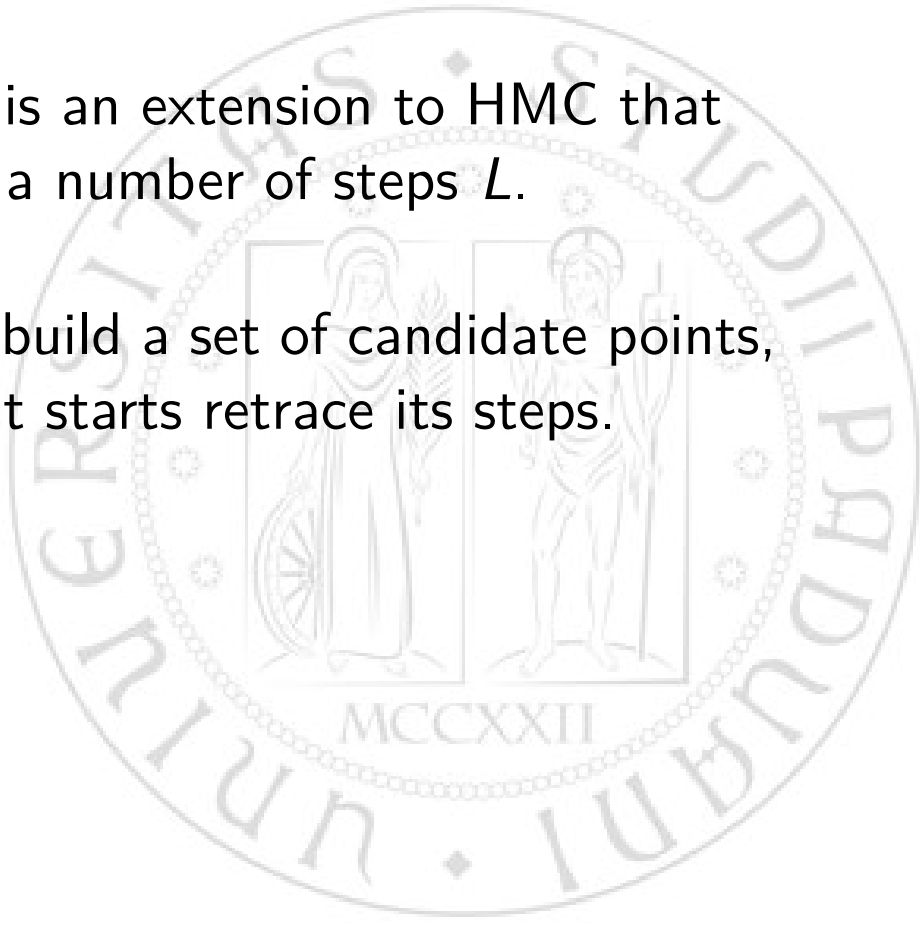
HMC is an *MCMC* algorithm that avoids random walk behavior and sensitivity to correlated parameters.
However, it requires the tuning of two user-specified parameters: step size ϵ and number of steps L.

$\Downarrow$

The *No-U-Turn Sampler* (NUTS) is an extension to HMC that eliminates the need to set a number of steps L.

$\Downarrow$

NUTS uses a recursive algorithm to build a set of candidate points, stopping automatically when it starts retrace its steps.



No U-Turn Sampler

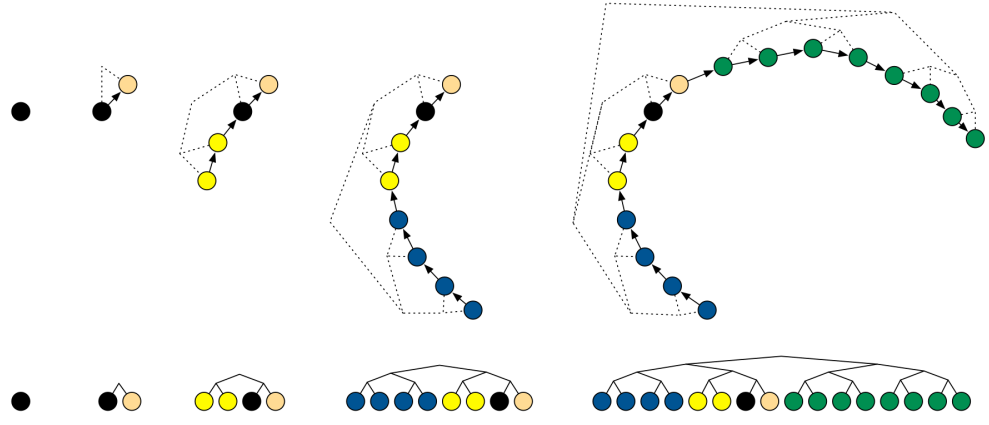
HMC is an *MCMC* algorithm that avoids random walk behavior and sensitivity to correlated parameters.
However, it requires the tuning of two user-specified parameters: step size ϵ and number of steps L.

$\Downarrow$

The *No-U-Turn Sampler* (NUTS) is an extension to HMC that eliminates the need to set a number of steps L.

$\Downarrow$

NUTS uses a recursive algorithm to build a set of candidate points, stopping automatically when it starts retrace its steps.



Hamiltonian Monte Carlo

HMC generates proposals with high acceptance probability by suppressing the random walk behavior, but it comes with a price:

Hamiltonian Monte Carlo

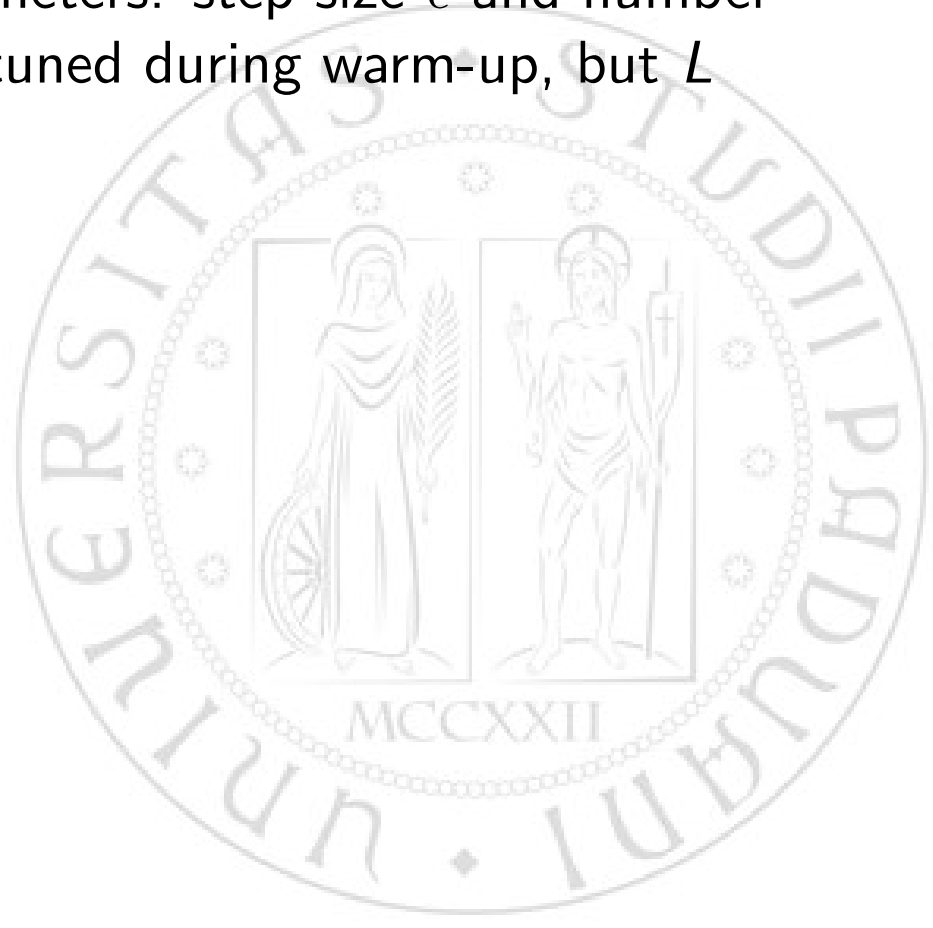
HMC generates proposals with high acceptance probability by suppressing the random walk behavior, but it comes with a price:

- Comuputing the gradient of the log-posterior at each step.

Hamiltonian Monte Carlo

HMC generates proposals with high acceptance probability by suppressing the random walk behavior, but it comes with a price:

- Comuputing the gradient of the log-posterior at each step.
- Tuning two user-specified parameters: step size ϵ and number of steps L. Tipically, ϵ can be tuned during warm-up, but L can require costly tuning runs.



Hamiltonian Monte Carlo

HMC generates proposals with high acceptance probability by suppressing the random walk behavior, but it comes with a price:

- Comuputing the gradient of the log-posterior at each step.
- Tuning two user-specified parameters: step size ϵ and number of steps L. Tipically, ϵ can be tuned during warm-up, but L can require costly tuning runs.

Algorithm 1 Hamiltonian Monte Carlo

Given θ^0, ϵ, L, $\mathcal{L}$, M:
for $m = 1$ to M **do**
  Sample $r^0 \sim \mathcal{N}(0, I)$.
  Set $\theta^m \leftarrow \theta^{m-1}, \tilde{\theta} \leftarrow \theta^{m-1}, \tilde{r} \leftarrow r^0$.
  for $i = 1$ to L **do**
    Set $\tilde{\theta}, \tilde{r} \leftarrow \text{Leapfrog}(\tilde{\theta}, \tilde{r}, \epsilon)$.
  end for
  With probability $\alpha = \min\left\{1, \frac{\exp\{\mathcal{L}(\tilde{\theta}) - \frac{1}{2}\tilde{r}\cdot\tilde{r}\}}{\exp\{\mathcal{L}(\theta^{m-1}) - \frac{1}{2}r^0\cdot r^0\}}\right\}$, set $\theta^m \leftarrow \tilde{\theta}$, $r^m \leftarrow -\tilde{r}$.
end for

function Leapfrog(θ, r, ϵ)
Set $\tilde{r} \leftarrow r + (\epsilon/2)\nabla_\theta \mathcal{L}(\theta)$.
Set $\tilde{\theta} \leftarrow \theta + \epsilon\tilde{r}$.
Set $\tilde{r} \leftarrow \tilde{r} + (\epsilon/2)\nabla_\theta \mathcal{L}(\tilde{\theta})$.
return $\tilde{\theta}, \tilde{r}$.

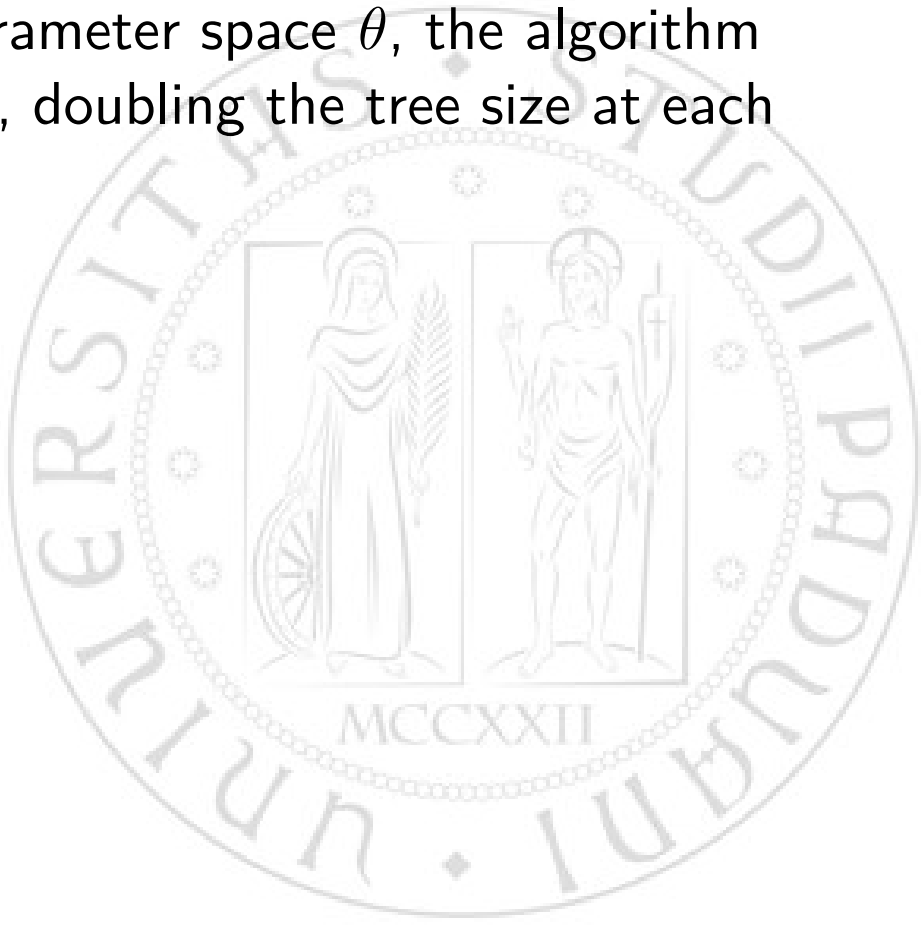
No U-Turn HMC

We want a sampler able to retain the HMC suppression on random walk behavior, but without the need to tune the number of Leap Frog steps L.

No U-Turn HMC

We want a sampler able to retain the HMC suppression on random walk behavior, but without the need to tune the number of Leap Frog steps L.

- Starting from a point in the parameter space θ, the algorithm builds a set of candidate points, doubling the tree size at each iteration.



No U-Turn HMC

We want a sampler able to retain the HMC suppression on random walk behavior, but without the need to tune the number of Leap Frog steps L.

- Starting from a point in the parameter space θ, the algorithm builds a set of candidate points, doubling the tree size at each iteration.
- To detect when to stop, the algorithm checks if the trajectory starts to turn back on itself: given a proposed point $\tilde{\theta}$, the algorithm checks if the dot product of the momentum vectors at θ and $\tilde{\theta}$ is negative.
  $(\theta - \tilde{\theta}) \cdot \frac{d}{dt}(\theta - \tilde{\theta}) = (\theta - \tilde{\theta}) \cdot \tilde{r} < 0$

No U-Turn HMC

We want a sampler able to retain the HMC suppression on random walk behavior, but without the need to tune the number of Leap Frog steps L.

- Starting from a point in the parameter space θ, the algorithm builds a set of candidate points, doubling the tree size at each iteration.
- To detect when to stop, the algorithm checks if the trajectory starts to turn back on itself: given a proposed point $\tilde{\theta}$, the algorithm checks if the dot product of the momentum vectors at θ and $\tilde{\theta}$ is negative.
  $(\theta - \tilde{\theta}) \cdot \frac{d}{dt}(\theta - \tilde{\theta}) = (\theta - \tilde{\theta}) \cdot \tilde{r} < 0$
- To preserve time reversibility, NUTS runs the Hamiltonian dynamics in both forward and backward directions.

Probability slicing

To simplify the derivation and implementation of the algorithm, NUTS uses a slice variable u with conditional distribution

$$p(u|\theta, r) = \mathcal{U}(u; [0, exp\mathcal{L}(\theta) - \tfrac{1}{2} r \cdot r]).$$

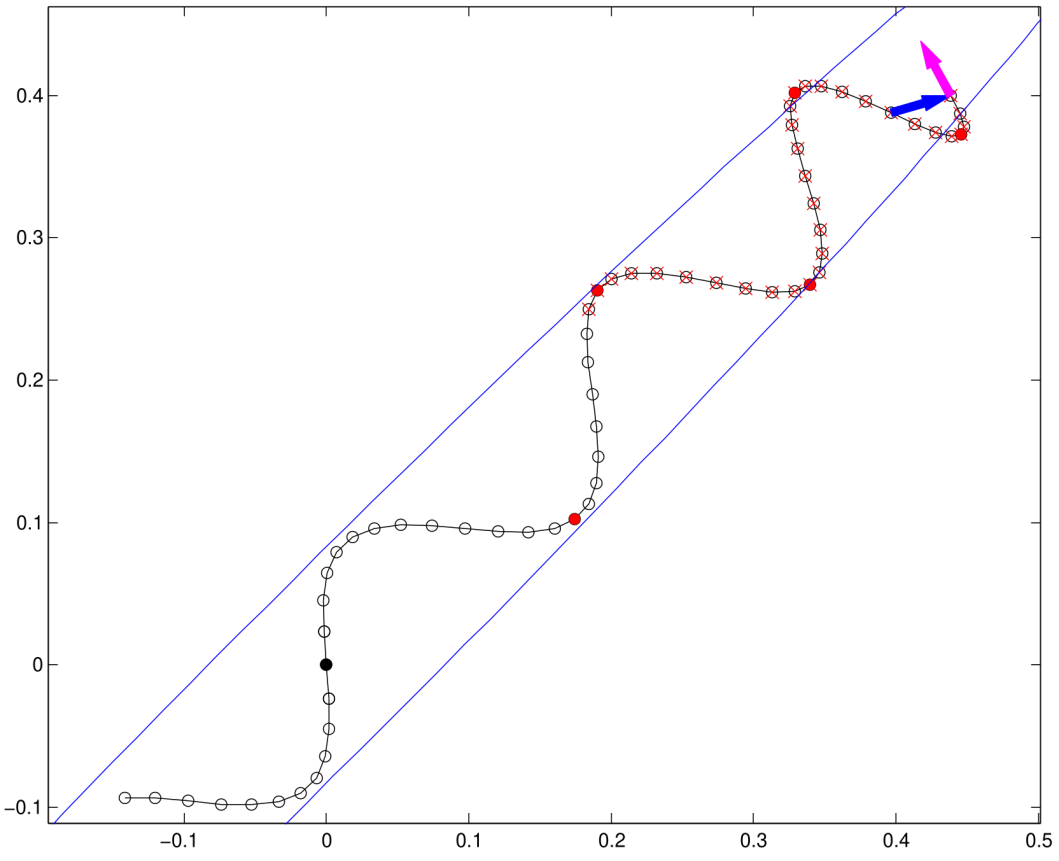
This way $p(\theta, r|u) = \mathcal{U}(\theta, r; \theta', r'|exp\mathcal{L}(\theta) - \tfrac{1}{2} r \cdot r \geq u)$.

Probability slicing

To simplify the derivation and implementation of the algorithm, NUTS uses a slice variable u with conditional distribution

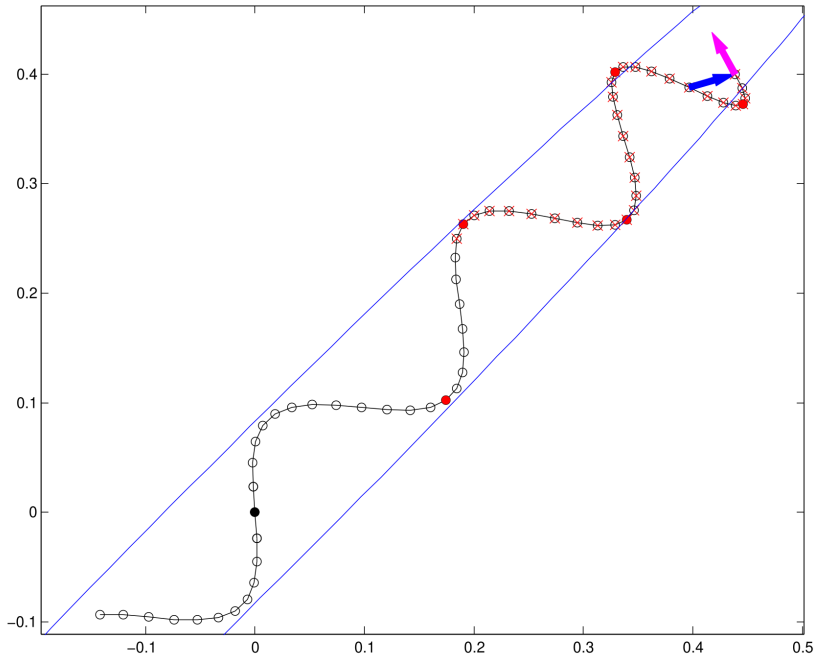
$$p(u|\theta, r) = \mathcal{U}(u; [0, exp\mathcal{L}(\theta) - \frac{1}{2} r \cdot r]).$$

This way $p(\theta, r|u) = \mathcal{U}(\theta, r; \theta', r'|exp\mathcal{L}(\theta) - \frac{1}{2} r \cdot r \geq u)$.



High-levl NUTS algorithm

- The algorithm resamples $u|\theta, r$ at each iteration,
- uses Leapfrog integrator to trace a path forward and backwards first 1 step,
- then doubling for successive iterations.
- This doubling process implicitly builds a balanced binary tree whose leaf nodes correspond to position-momentum states.
- The doubling is halted when the subtrajectory starts to double back on itself.



Example of a NUTS trajectory, with the slice variable u.